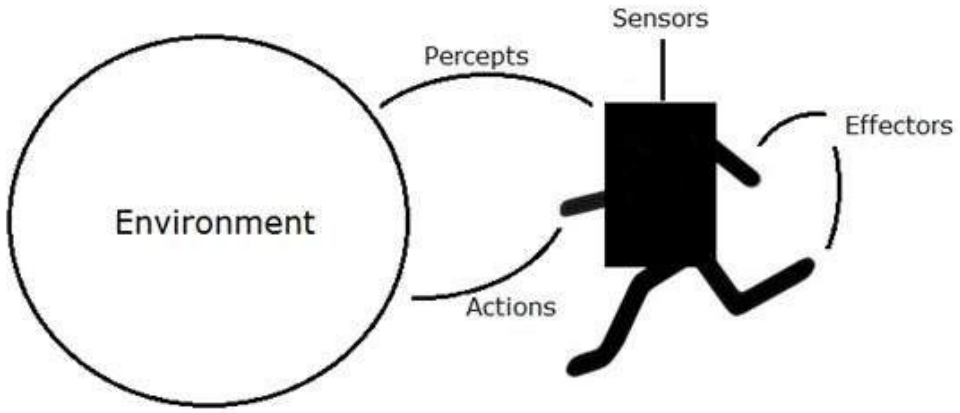
Logical Agents and Planning

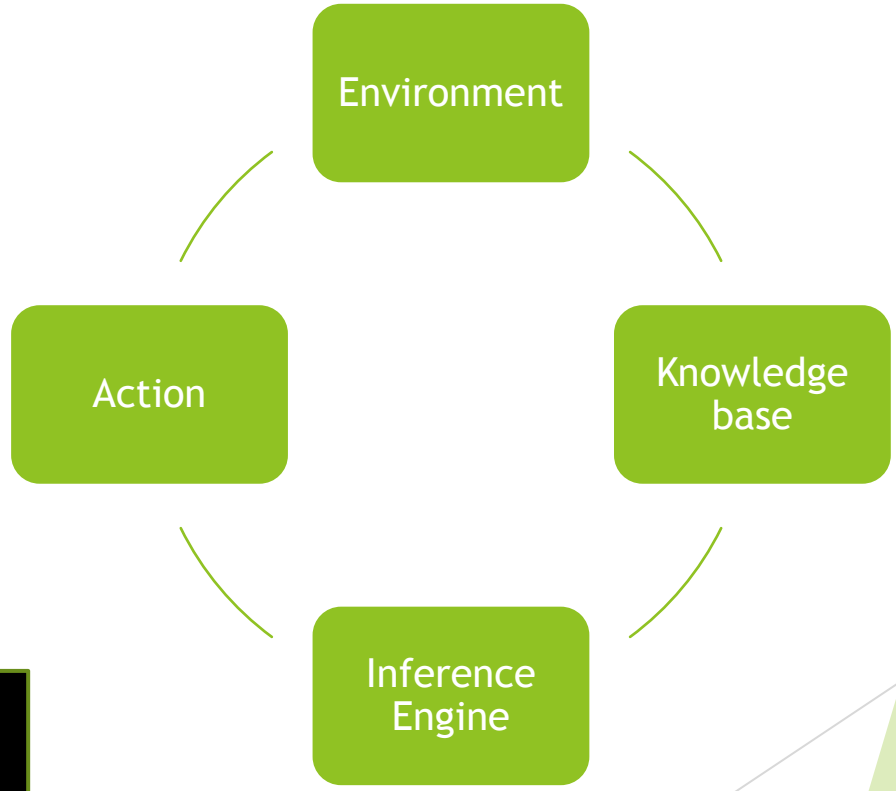
Logical Agents

- A logical agent works by deducing what to do given a knowledge base of words about the world.
- Axioms are the general information about how the universe works combine with percept sentences extracted from the agent's experience in a specific reality to form the knowledge base.
- It uses formal logic to represent its knowledge about the world and to reason about what actions to take.
- The word *logical* here means its decision-making is grounded in explicit, verifiable symbolic reasoning, not just pattern matching or reflexes.
- A logical agent needs three things:
  - A way to represent what it believes about the world
  - A way to reason about how actions change the world
  - A way to plan a sequence of actions to reach a goal



Logical Agents

- **Agent = Perception + Reasoning + Action**
- Every agent follows the basic loop
- Inference Engine: applies logical rules to the KB to derive new conclusions and decide what action to take. Two operations it performs:
  - TELL: add a new fact to the KB (after perceiving something)
  - ASK: query the KB to decide what to do next.




```
Perceive: Dirty(RoomA) → TELL(KB, Dirty(RoomA))
Decide:   ASK(KB, WhatShouldIDo?) → Clean(RoomA)
Act:      Execute Clean(RoomA)
TELL(KB, ¬Dirty(RoomA))   ← update after acting
```

% KB Facts

Temperature(18).

DesiredTemp(22).

HeaterStatus(off).

% KB Rules

TurnOnHeater ← Temperature(t) ∧ DesiredTemp(d) ∧ t < d ∧ HeaterStatus(off)

TurnOffHeater ← Temperature(t) ∧ DesiredTemp(d) ∧ t >= d ∧ HeaterStatus(on)

% Inference

ASK: What action should I take?

→ Temperature(18) < DesiredTemp(22) ∧ HeaterStatus(off)

→ Fires rule: TurnOnHeater

→ Agent acts: switches heater ON

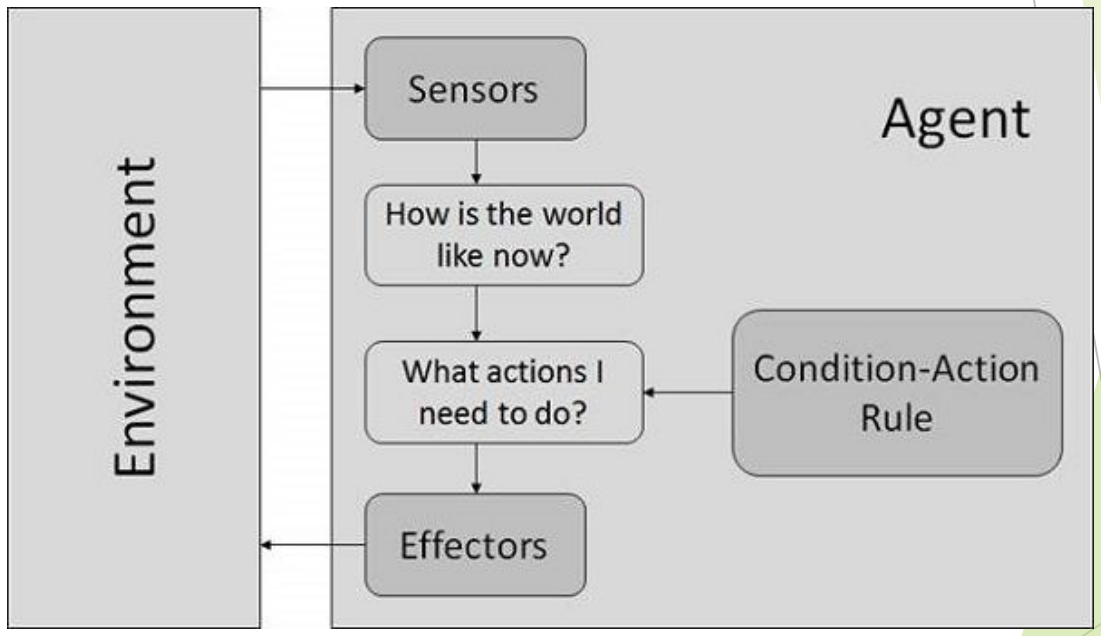
→ TELL(KB, HeaterStatus(on))

Types of Agents

Type	How it decides	Example
Simple reflex	If-then rules on current percept only	Thermostat (fixed threshold)
Model-based reflex	Keeps internal state, still rule-based	Roomba with room map
Goal-based	Considers future states to reach a goal	GPS navigator
Logical agent	Uses a full KB + inference engine	BDI agent, theorem prover

Simple Reflex Agents

- Actions on current percept only.
- No memory of the past
- No model of the world
- Just a direct mapping from what it sees right now to what it does.
- If X is perceived, the do Y.
- Condition-Action Rule
  - Percept → Condition Matching → Action




```
def simple_reflex_agent(percept):
    rule = match_rule(percept, rules)
    action = rule.action
    return action
```

Simple Reflex Agents

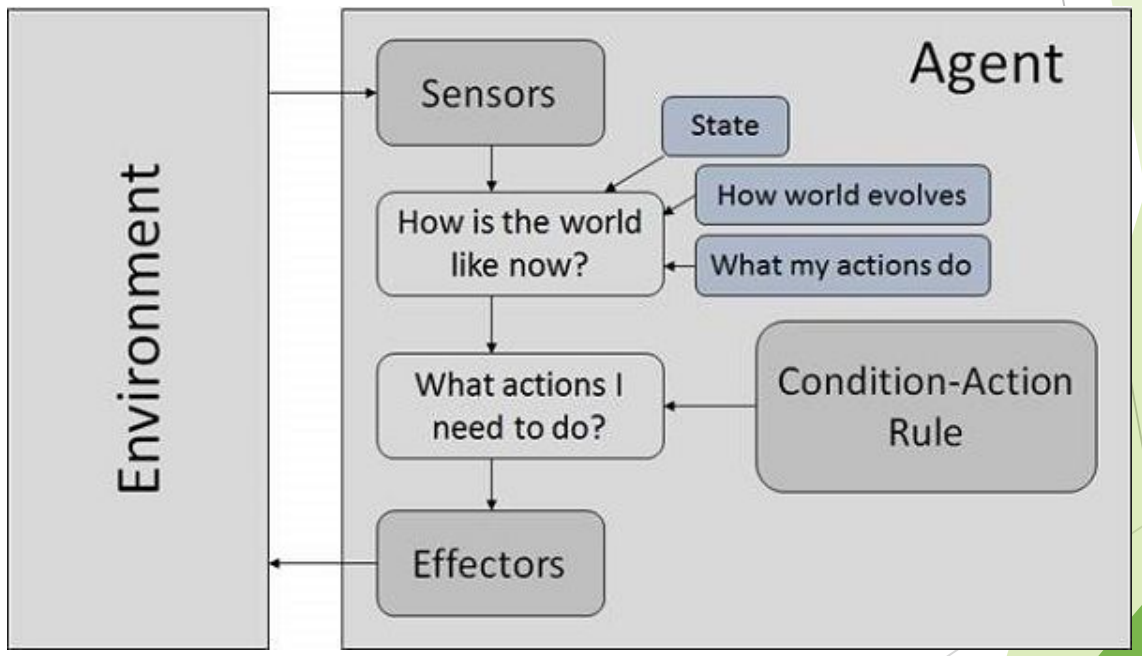
- **Condition-Action Rules**
- These are the agent's entire "knowledge."
- Example for a vacuum cleaner agent:

Condition (Percept)	Action
Current location is dirty	Clean
Current location is clean	Move forward
Bump into wall	Turn right / Turn left

- **Limitations:**
  - No memory: can't handle sequences or history
  - Breaks in partially observable environments: if it can't see the full state, it may loop or act wrongly
  - Inflexible: can't adapt to situations not covered by its rules

Model Based Reflex Agents

- It solves the biggest weakness of the simple reflex agent: **no memory.**
- Remember the past. Build a model of the world. Then act.
- The agent keeps an internal representation of the world that gets updated with every new percept.
- It uses this model *plus* the current percept to decide what to do.



Model Based Reflex Agents

- Percept → Update Internal State → Condition Matching → Action
- State: what the agent remembers about the world
- Model: knowledge about how the world works and how actions affect it
- **The Two Parts of the Model**
  - Transition model: How does the world change over time on its own?
  - Sensor model: What does current percept tells about the world?
  - Together, these let the agent fill in what it **can't directly observe**.

```
def model_based_reflex_agent(percept, state, model, rules):
    state = update_state(state, percept, model)
    rule = match_rule(state, rules)
    action = rule.action
    return action
```

Model Based Reflex Agents

- Vacuum World (Partial Observability):
  - Imagine the vacuum can only sense its **current room**, it can't see the other room.
  - A simple reflex agent would have no idea whether the other room is dirty. A model-based agent remembers:
    - state = { room_A: 'clean', room_B: 'unknown', current_location: 'A' }
  - After cleaning A and moving to B, it updates:
    - state = { room_A: 'clean', room_B: 'dirty', current_location: 'B' }
  - {room_A: 'clean'} is stored in its internal state, it doesn't need to see room A again to know it's clean.
- Self-Driving Car:
  - The internal state tracks: road conditions, positions of other cars, previous signals, things that vanish from perception but still matter.

What it perceives	What it can't see right now
Current speed, nearby cars	Cars that moved out of sensor range
Traffic light ahead	Whether light was red 2 seconds ago

Model Based Reflex Agents

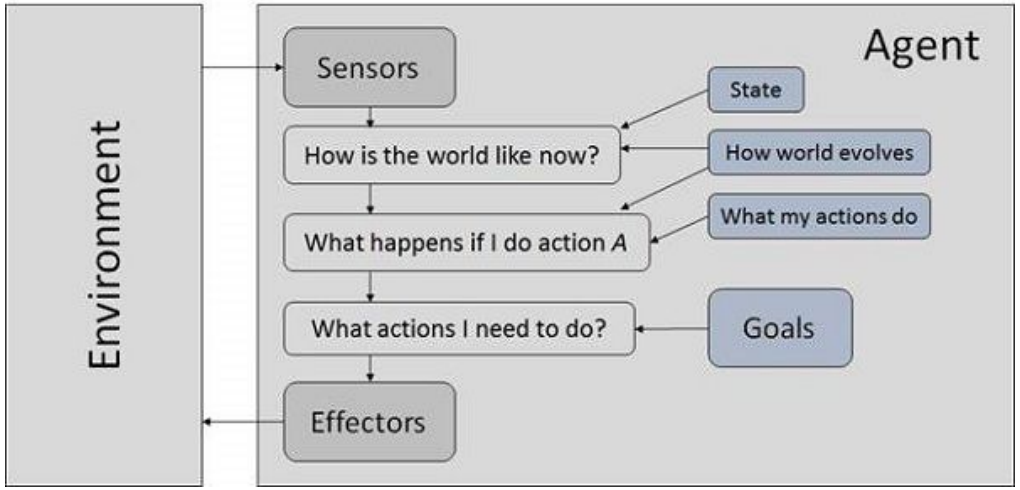
- **Strengths over Simple Reflex:**
  - **Handles partial observability:** works even when the full world isn't visible
  - **Context-aware:** past events influence current decisions
  - **More robust:** doesn't blindly react to just the current moment
- Limitations:
  - **Still rule-based:** no goals, no planning ahead
  - **Model can be wrong:** if the transition/sensor model is inaccurate, the internal state drifts from reality
  - **No optimization:** it doesn't ask *"what's the best action?"*, only *"what rule fires?"*

Model Based Reflex Agents

```
rooms = {'A': 'dirty', 'B': 'dirty'}
state = {'location': 'A', 'A': 'unknown', 'B': 'unknown'}
def agent(state, rooms):
    loc = state['location']
    percept = rooms[loc]       # sense current room
    state[loc] = percept       # update memory
    if percept == 'dirty':
        rooms[loc] = 'clean'
        action = 'suck'
    else:
        state['location'] = 'B' if loc == 'A' else 'A'
        action = 'move'
    return action, loc
for i in range(4):
    action, loc = agent(state, rooms)
    print(f"Step {i+1}: loc={loc}, action={action}, memory={{'A': state['A'], 'B': state['B']}}")
```

Goal Based Agents

- Don't just react. Think ahead. Choose actions that lead to the goal.
- It doesn't just react to the current state, it asks **"what do I want to achieve?"** and plans actions to get there.
- The agent has:
  - A model of the world (like model-based)
  - A goal (a desired state it wants to reach).
  - The ability to search/plan a sequence of actions to reach that goal
- Percept → Update State → Check Goal → Search/Plan → Action



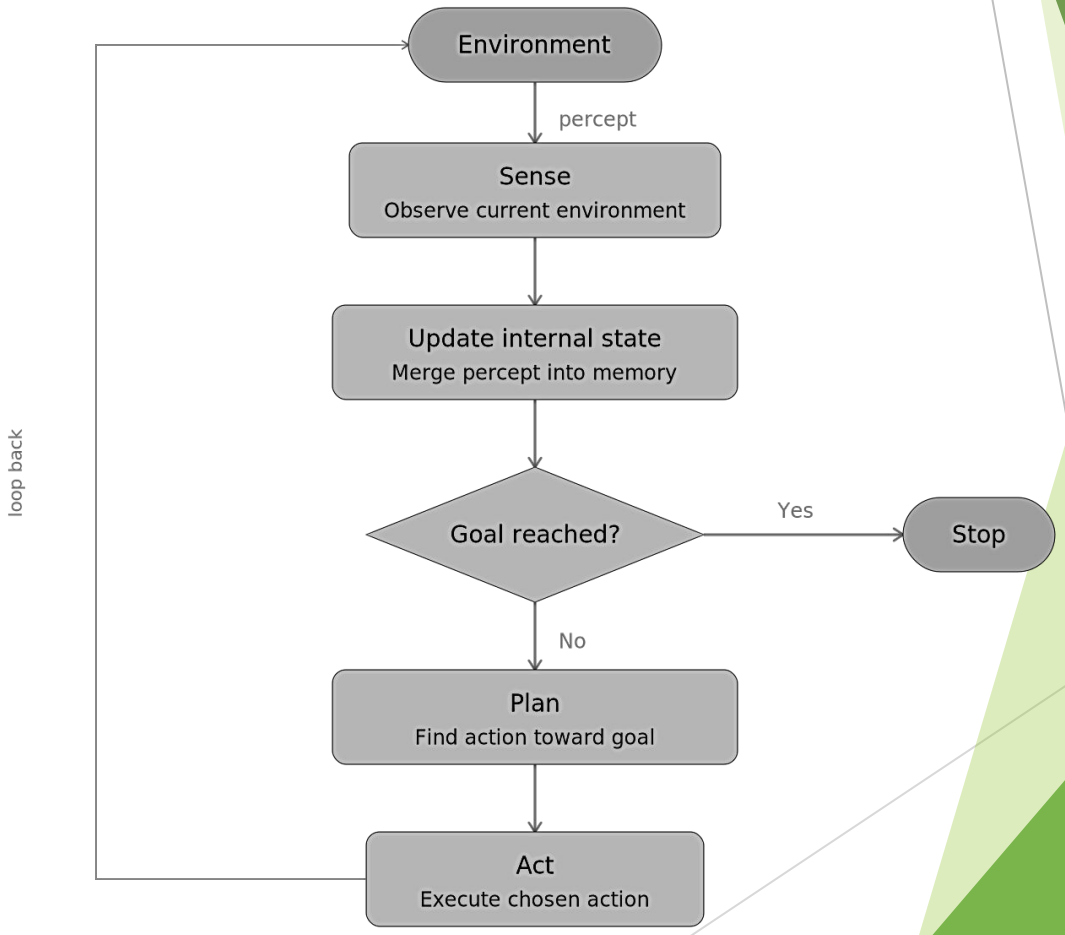
Vacuum World with a Goal

```
rooms = {'A': 'dirty', 'B': 'dirty'}
goal  = {'A': 'clean', 'B': 'clean'}

state = {'location': 'A', 'rooms': {'A': 'unknown', 'B': 'unknown'}}

def plan(state, goal):
    for room, status in state['rooms'].items():
        if status != 'clean':          # found a dirty room
            if state['location'] != room:
                state['location'] = room
                return 'move to {room}'
            else:
                return ‘clean {room}'
    return 'stop'                      # goal reached

def agent(state, rooms, goal):
    loc = state['location']
    state['rooms'][loc] = rooms[loc]   # sense & update memory

    if state['rooms'] == goal:
        return 'stop', state
    action = plan(state, goal)
    if ‘clean' in action:
        rooms[loc] = 'clean'
        state['rooms'][loc] = 'clean'

    return action, state

for i in range(5):
    action, state = agent(state, rooms, goal)
            print(f"Step    {i+1}:    loc={state['location']},    action={action},
rooms={state['rooms']}")
if action == 'stop':
        break
```

- Step 1: loc=A, action=clean A, rooms={'A': 'clean', 'B': 'unknown'}
- Step 2: loc=A, action=move to B, rooms={'A': 'clean', 'B': 'unknown'}
- Step 3: loc=B, action=clean B, rooms={'A': 'clean', 'B': 'clean'}
- Step 4: loc=B, action=stop, rooms={'A': 'clean', 'B': 'clean'}
- With a reflex agent, changing behavior means **rewriting rules**. With a goal-based agent, you just **change the goal**:
- goal = {'A': 'clean', 'B': 'dirty'} # only clean room A

Key Difference from Model-Based Reflex

	Model-Based Reflex	Goal-Based
Has internal state	YES	YES
Has goal	NO	YES
Plans ahead	NO	YES
Knows **when to stop**	NO	YES
Flexible to new goals	NO	YES



Planning

- One of the most useful ways that an intelligent agent can take advantage of the knowledge it has and its ability to reason about actions and their consequences.
  In the real world, because our actions are not totally guaranteed to have certain effects and because we simply cannot know everything there is to know about a situation, planning is usually an uncertain initiative.
- planning in the real world involves trying to determine what future states of the world will be like, but also observing the world as plans are being executed and re-planning as necessary.
- Make sure necessary basic capabilities are available.

Planning Situation Calculus

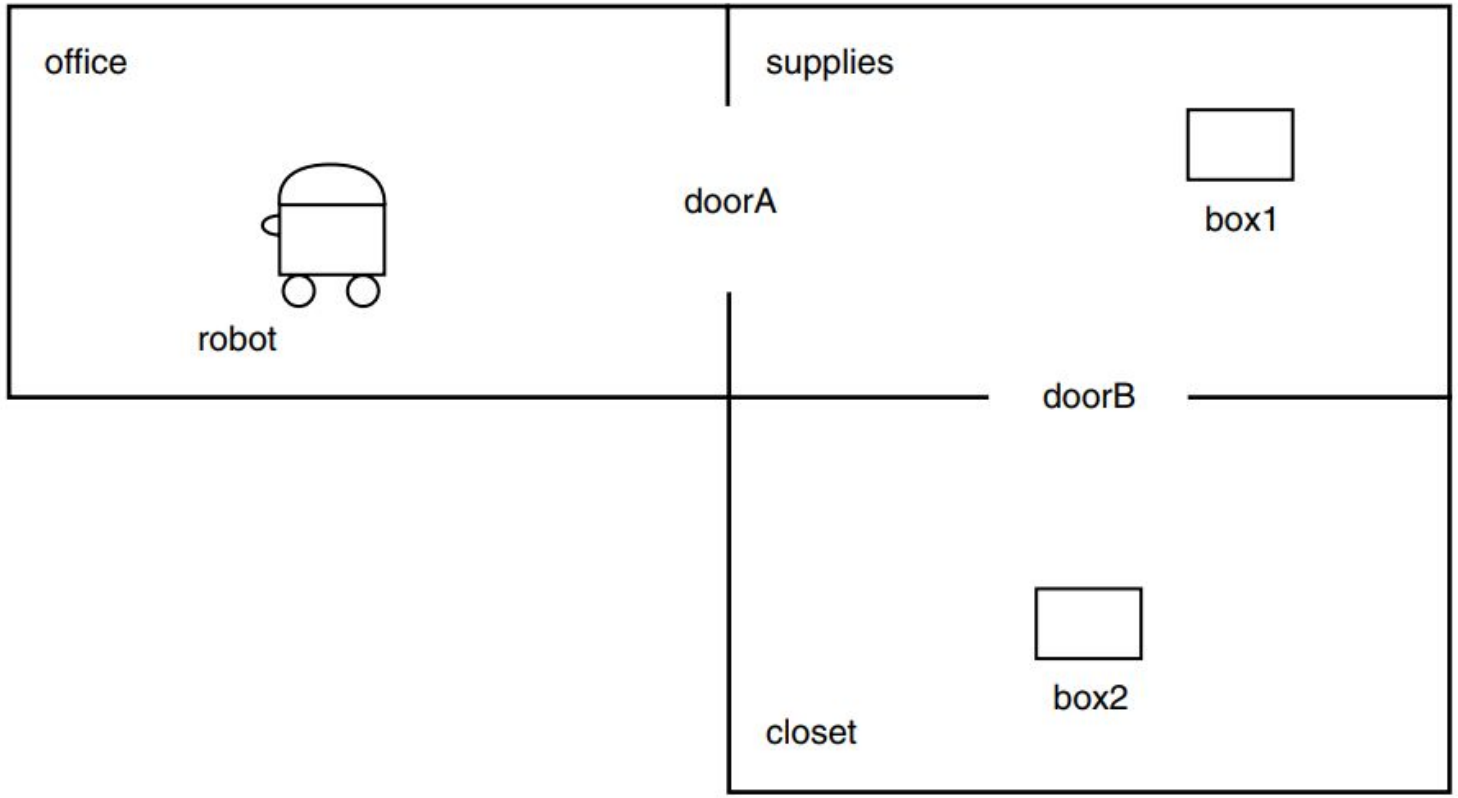
- Goal Formula:

$$\text{KB} \models Goal(do(\vec{a}, S_0)) \wedge Legal(do(\vec{a}, S_0))$$

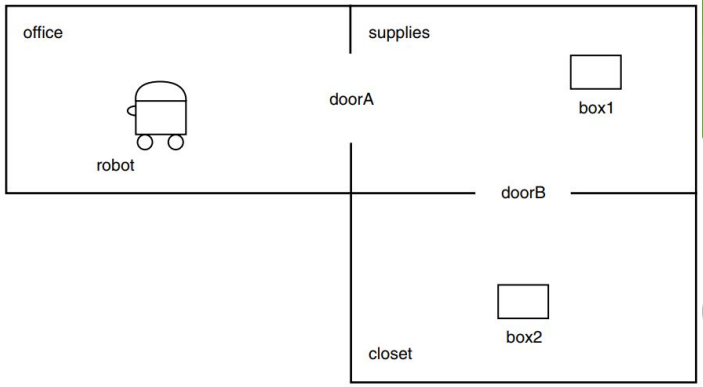
- The Knowledge Base entails that: executing action sequence ā from initial situation S_0 achieves the Goal and is Legal.
- $KB \models some - fact$
  - Given everything we know (KB), we can conclude some_fact is true.
- "All humans are mortal. Mr.X is human." ⊨ "Mr.X is mortal."
- given a goal formula, we wish to find a sequence of actions such that it follows from what is known that
  1. the goal formula will hold in the situation that results from executing the actions in sequence starting in the initial state, and
  2. it is possible to execute each action in the appropriate situation (that is, each action's preconditions are satisfied).

$$\text{KB} \models \exists s.\ Goal(s) \wedge Legal(s)$$

Planning Situation Calculus

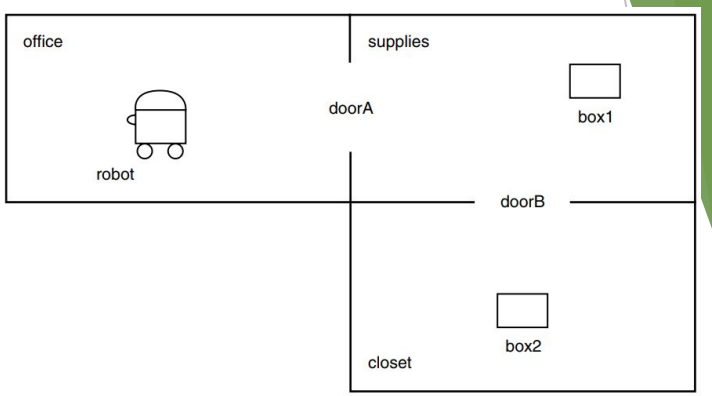


Planning Situation Calculus



- A robot can roll from room to room, possibly pushing objects through doorways between the rooms.
- In such a world, there are two actions:
  - $pushThru(x, d, r_1, r_2)$
  - in which the robot pushes object x through doorway d from room $r1$ to $r2$
  - $goThru(d, r_1, r_2)$
  - in which the robot rolls through doorway d from room $r1$ to $r2$.
- To be able to execute either action, *d* must be the doorway connecting *r*1 and *r*2, and the robot must be located in *r*1.
- After successfully completing either action, the robot ends up in room *r*2.
- In addition, for the action pushThru, the object *x* must be located initially in room *r*1, and will also end up in room *r*2.

Planning Situation Calculus



- We can formalize these properties of the world in the situation calculus using the following two precondition axioms:
  - *Poss*(goThru(*d*, *r*1, *r*2), *s*) ≡ Connected(*d*, *r*1, *r*2) ∧ InRoom(robot, *r*1, *s*)
  - *Poss*(pushThru(*x*, *d*, r_1, r_2), *s*) ≡ Connected(*d*, *r*1, *r*2) ∧ InRoom(robot, *r*1, *s*) ∧ InRoom(*x*, *r*1, *s*).
  - $InRoom(x,r,do(a,s)) \equiv \sqcap(x,a,r) \vee (InRoom(x,r,s) \wedge \neg\exists r'.(r \neq r') \wedge \sqcap(x,a,r'))$
- where $\sqcap(x,a,r)$ is the formula
  - $x = robot \wedge \exists d \exists r1.a = goThru(d,r1,r)$
    $\vee\ x = robot \wedge \exists d \exists r1 \exists y.a = pushThru(y,d,r1,r) \vee \exists d \exists r1.a = pushThru(x,d,r1,r).$
- the robot is in room *r* after an action if that action was either a goThru or a pushThru to *r*, or the robot was already in *r* and the action was not a goThru or a pushThru to some other *r*′.
- For any other object, the object is in room *r* after an action if that action was a pushThru to *r* for that object, or the object was already in *r* and the action was not a pushThru to some other *r*′ for that object.

Planning Situation Calculus

- $[x \neq robot, a \neq goThru(d, r1, r2), InRoom(x, r2, do(a, s))]$
- $[x \neq robot, a \neq pushThru(y, d, r1, r2), InRoom(x, r2, do(a, s))]$
- $[a \neq pushThru(x, d, r1, r2), InRoom(x, r2, do(a, s))]$
- $[\neg InRoom(x, r, s), x = robot, a = pushThru(x, t0, t1, t2), InRoom(x, r, do(a, s))]$
- $[\neg InRoom(x, r, s), a = goThru(t3, t4, t5),$
  $a = pushThru(t6, t7, t8, t9), InRoom(x, r, do(a, s))]$

Planning Situation Calculus: *successor state axiom*

- $[x \neq robot, a \neq goThru(d, r1, r2), InRoom(x, r2, do(a, s))]$
  - If x is not the robot, AND the action is not goThru, THEN x ends up in r2.
  - non-robot objects don't move by themselves, so if x was already in r2, it stays there regardless of what action happens.
- $[x \neq robot,\ a \neq pushThru(x,\ d,\ r1,\ r2),\ InRoom(x,\ r2,\ do(a,\ s))]$
  - If x is not the robot AND the action is not pushThru, THEN x stays in r2.
  - This covers the box/object case, a non-robot object only moves if it is explicitly pushed. Otherwise it stays put.
- $[a \neq pushThru(x, d, r1, r2), InRoom(x, r2, do(a, s))]$
  - If the action is not pushThru applied to x specifically, THEN x stays in r2.
  - This narrows it further, even if something is being pushed, x stays unless x itself is the object being pushed.

Planning Situation Calculus: *successor state axiom*

- $[\neg InRoom(x, r, s), x = robot, a = pushThru(x, t0, t1, t2), InRoom(x, r, do(a, s))]$
  - If x was NOT in room r before, BUT x is the robot AND the action is pushThru(x, ...), THEN x IS in room r after action a.
  - This is the robot movement clause – when the robot performs pushThru, it moves itself from one room to another. The t0, t1, t2 are the door and rooms involved.
- $[\neg InRoom(x, r, s), a = goThru(t3, t4, t5), a = pushThru(t6, t7, t8, t9), InRoom(x, r, do(a, s))]$
  - If x was NOT in room r before, AND the action is either goThru OR pushThru, THEN x IS in room r after action a.
  - This handles the frame problem, it says: if x wasn't there before and neither a goThru nor a pushThru moved it there, then it can't be there now.

Planning Situation Calculus: *successor state axiom*

Clause	Covers
1	Non-robot objects are unaffected by goThru
2	Non-robot objects are unaffected by pushThru on others
3	Object x stays unless x itself is pushed
4	Robot moves to new room when it performs pushThru
5	Frame rule — object only appears in room if an action moved it there

Belief-Desire-Intention (BDI)

- BDI is a model for how a rational agent thinks and acts, inspired by how **humans** reason.
- Belief:
  - What does the agent think is true?
  - Room A is dirty
- Desire:
  - What does the agent want to achieve?
  - All rooms clean
- Intention:
  - What has the agent committed to doing?
  - Clean room A first
- having a desire doesn't mean you act on it immediately.
- commit to an intention, then follow through (even if new information arrives)
- Example:
  - **Believe** something about the world, "it's raining outside“
  - **Desire** an outcome, "I want to stay dry“
  - **Intend** a specific plan, "I'll take an umbrella"

Belief-Desire-Intention (BDI)

- Belief (What do I know?):
  - The agent's internal model of the world.
  - It may be **incomplete or wrong**, it's what the agent *thinks* is true, not necessarily what *is* true.

```
beliefs = {
    'room_A': 'dirty',
    'room_B': 'clean',
    'location': 'A',
    'battery': 'full'
}
```

  - Beliefs get **updated** when the agent perceives new information.

Belief-Desire-Intention (BDI)

- **Desire (What do I want?)**
  - All the goals the agent would like to achieve.
  - Desires can be **multiple and conflicting** — the agent doesn't act on all of them at once.

```
desires = [
    'all rooms clean',
    'save battery',
    'finish quickly'
]
```

  - save battery and finish quickly conflict. The agent has to choose.

Belief-Desire-Intention (BDI)

- Intention (What am I committed to?)
  - The desire the agent has selected and committed to acting on.
  - This is what separates BDI from simpler agents.
  - Without commitment, an agent would re-deliberate every single step, *"should I still clean A? or switch to B? or save battery?"*, and waste all its time thinking instead of acting.
  - So, here intention means: I have decided. I will do this. I won't second-guess myself unless I have a good reason.

    ```
    intentions = ['clean room_A']
    ```

  - Committed: will follow through
  - Once committed, the agent **sticks to this plan** and doesn't keep re-evaluating unless something important changes.
  - **When Does the Agent Drop an Intention?**
    - Goal already achieved
    - Goal becomes impossible
    - Better option appears